Le-QLN — mise en ligne du serveur

Multijoueur, comptes, classements : Render, Cloudflare, Resend · 10 novembre 2026

Le code est prêt (dossier **server/** du dépôt) : relais multijoueur WebSocket, API comptes / progression / scores / classements / mots contestés, tables PostgreSQL créées automatiquement (préfixe qln_). Il reste à l'héberger et à brancher l'appli dessus.

1. Qui fait quoi

Service	Rôle pour Le-QLN	Quand	Coût
Render — Web Service	héberge server/ (multijoueur + API)	**aujourd'hui**	Free pour tester ; Starter 7 $/mois en public
Render — PostgreSQL dédiée	comptes, scores, classements	**aujourd'hui** (créée par le Blueprint)	≈ 6 $/mois
GitHub Actions	tests, vérifications, appli Android	déjà en place	gratuit (dépôt public)
Google Cloud (OAuth)	bouton « Se connecter avec Google »	aujourd'hui, facultatif	gratuit
Cloudflare	nom de domaine + DNS (site, api., e-mails)	dès que le nom est choisi	≈ 10-12 €/an
Resend	e-mails « mot de passe oublié »	après le domaine	gratuit (3 000 e-mails/mois)
GitHub Pages	héberge l'appli (déjà en place)	—	gratuit

2. Aujourd'hui : Render en un clic (≈ 5 minutes)

Tout le service est décrit dans le fichier **render.yaml** du dépôt (« Blueprint ») : serveur, base de données dédiée, variables. Rien à recopier.

1. Render → **New → Blueprint** → choisir le dépôt **Kiki78test/Le-QLN**.

2. Render affiche ce qu'il va créer : le service **le-qln** (Frankfurt, gratuit pour les essais) et la base **le-qln-db** (PostgreSQL dédiée, ≈ 6 $/mois). Cliquer **Apply**.

3. Attendre « Live » (3-5 min), puis ouvrir **https://le-qln.onrender.com/sante** : la réponse doit contenir **"api":true**.

4. Envoyer cette adresse à Claude : il renseigne config.js et pousse. Les comptes, classements et salons en ligne s'activent.

Pourquoi une base dédiée plutôt que celle d'Airmony : une restauration, une panne ou une saturation de l'une ne touche pas l'autre ; sauvegardes et coûts séparés ; le jeu peut être cédé ou déplacé seul. Coût : ≈ 6 $/mois.

Ensuite, tout est automatique

Quand	Ce qui se passe tout seul
Chaque envoi sur GitHub	**Tests** : serveur (comptes, scores, classements, salons) sur une base de test, puis l'appli dans Chromium (FR/EN, téléphone/ordinateur, carte jouée, plateau, création de compte).
Tests réussis + server/ modifié	**Render redéploie** le serveur (jamais si un test échoue).
Site mis en ligne (GitHub Pages)	**Version en ligne** : l'appli publiée est retestée.
Toutes les 6 heures	Le site, le serveur et la base répondent.
Chaque lundi	Contrôle de sécurité des dépendances du serveur.
mobile/ modifié	**Appli Android** de test fabriquée et publiée (Releases → « Appli Android (test) »).
Un échec	Croix rouge dans l'onglet Actions et **e-mail de GitHub**.

Appli Android de test : l'APK ouvre le site en ligne ; chaque mise à jour du jeu arrive donc sans réinstaller. Elle ne se réinstalle que si l'enveloppe change (rare). La connexion Google y est masquée (Google la refuse dans une appli de ce type) : e-mail + mot de passe.

C. Connexion Google (facultatif)

1. console.cloud.google.com → nouveau projet « Le-QLN » → *API et services* → *Écran de consentement OAuth* : externe, nom de l'appli, votre e-mail.
2. *Identifiants* → *Créer* → *ID client OAuth* → *Application Web*. Origines JavaScript autorisées : **https://kiki78test.github.io**.
3. Copier l'ID client (…apps.googleusercontent.com) dans la variable GOOGLE_CLIENT_ID de Render et l'envoyer à Claude (config.js).

D. Vérifications à faire ensemble

• Créer un compte sur téléphone, se connecter sur PC : la progression suit.
• Finir un défi du jour : le score apparaît dans Classements (monde et pays).
• Ouvrir un salon sur un appareil : il est visible et rejoignable depuis l'autre.
• Contester un mot valide : il est revérifié par le serveur et ajouté pour tous.

3. Ensuite : Cloudflare (quand le nom est choisi)

1. Cloudflare → *Domain Registration* → *Register* : acheter le domaine (prix coûtant).
2. DNS : **CNAME www → kiki78test.github.io** et les 4 enregistrements A de GitHub Pages pour la racine, en mode **DNS only** (nuage gris) pour laisser GitHub délivrer le certificat. Dans GitHub → Settings → Pages → Custom domain : le domaine, puis « Enforce HTTPS ».
3. DNS : **CNAME api → le-qln.onrender.com** (DNS only), puis Render → Settings → Custom Domains → api.votre-domaine.
4. Claude met à jour config.js (QLN_API_URL = https://api.votre-domaine), ORIGINES (render.yaml), le manifeste et l'ID Google (nouvelle origine).
5. Le domaine débloque aussi AdSense (version web) et Resend.

4. Ensuite : Resend (mot de passe oublié)

1. Créer le compte resend.com (gratuit) → *Domains* → *Add* : votre domaine. Resend affiche 3-4 enregistrements DNS (SPF, DKIM, MX de retour) : les ajouter dans Cloudflare (ou bouton « Auto configure » Cloudflare). Vérification en quelques minutes.
2. *API Keys* → *Create* (droit « Sending access ») → variable **RESEND_API_KEY** sur Render, plus **MAIL_FROM** = « Le-QLN <noreply@votre-domaine> ».
3. Claude code la route « mot de passe oublié » (lien valable 30 min, à usage unique) et l'écran correspondant. Avant le domaine, Resend ne peut écrire qu'à votre propre adresse : suffisant pour tester.

5. Budget mensuel

Poste	Test (aujourd'hui)	Public
Render Web Service	0 € (Free)	≈ 6,5 € (Starter 7 $)
PostgreSQL dédiée	≈ 5,5 € (6 $)	≈ 5,5 € (6 $)
Domaine Cloudflare	—	≈ 1 €/mois (≈ 10-12 €/an)
Resend, Google OAuth, GitHub Pages	0 €	0 €
Total	≈ 5,5 €/mois	≈ 13 €/mois

Limites connues : les scores sont calculés par l'appli (recalcul serveur du défi du jour prévu) ; comptes des moins de 15 ans à faire valider juridiquement ; le multijoueur synchronisé sur le plateau (partie complète à plusieurs) reste à développer après la mise en ligne — aujourd'hui les salons servent à se retrouver et lancer la partie.